

Experiment No.6
Implementation of Recursive Decent Parser (RDP)
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

Aim: Implementation of Recursive Decent Parser (RDP).

Objective: To develop an ability to design and develop the Analysis phase of Compiler.

Theory:

Recursive descent is a top-down parsing technique that constructs the parse tree from the top and the input is read from left to right. It uses procedures for every terminal and non-terminal entity. This parsing technique recursively parses the input to make a parse tree, which may or may not require back-tracking. But the grammar associated with it (if not left factored) cannot avoid back-tracking. A form of recursive-descent parsing that does not require any backtracking is known as predictive parsing. This parsing technique is regarded recursive as it uses context-free grammar which is recursive in nature.

Back-tracking

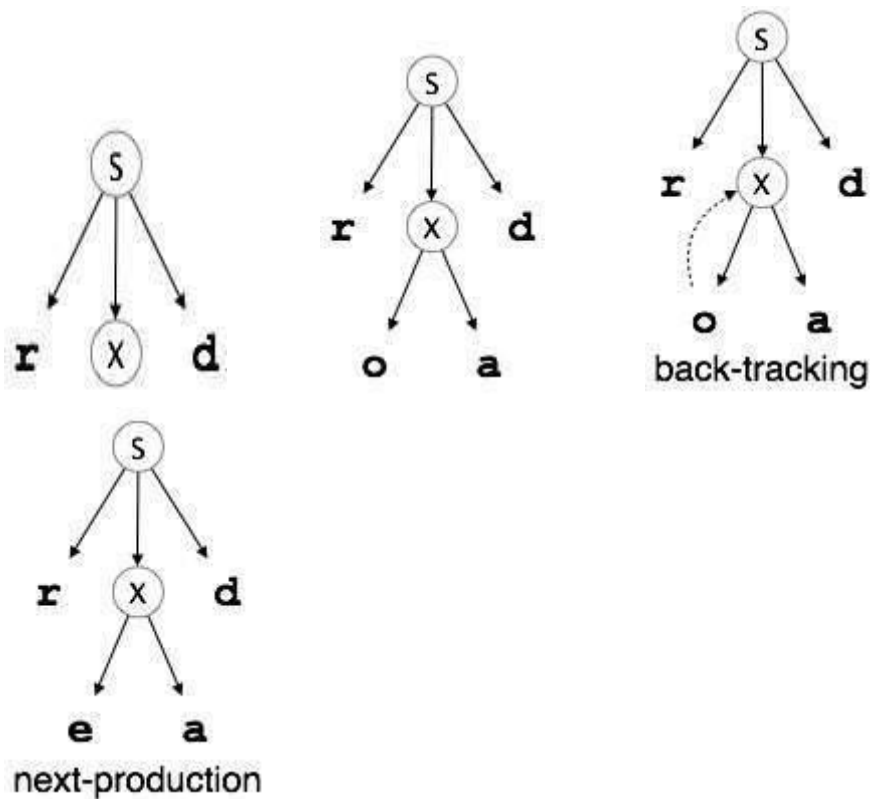
Top- down parsers start from the root node (start symbol) and match the input string against the production rules to replace them (if matched). To understand this, take the following example of CFG:

$$S \rightarrow rXd \mid rZd$$
$$X \rightarrow oa \mid ea$$
$$Z \rightarrow ai$$

For an input string: read, a top-down parser, will behave like this:

It will start with S from the production rules and will match its yield to the left-most letter of the input, i.e. 'r'. The very production of S ($S \rightarrow rXd$) matches with it. So the top-down parser advances to the next input letter (i.e. 'e'). The parser tries to expand non-terminal 'X' and checks its production from the left ($X \rightarrow oa$). It does not match with the next input symbol. So the top-down parser backtracks to obtain the next production rule of X, ($X \rightarrow ea$).

Now the parser matches all the input letters in an ordered manner. The string is accepted.



Code:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define SUCCESS 1
```

```
#define FAILED 0
```

```
// Function prototypes
```

```
int E(), Edash(), T(), Tdash(), F();
```

```
const char *cursor;
```

```
char string[64];
```

CSL601: System Programming and Compiler Construction Laboratory



```
int main() {
    puts("Enter the string");
    scanf("%s", string); // Read input from the user
    cursor = string;
    puts("");
    puts("Input          Action");
    puts("-----");

    // Call the starting non-terminal E
    if (E() && *cursor == '\0') { // If parsing is successful and the cursor has reached the
end
        puts("-----");
        puts("String is successfully parsed");
        return 0;
    }
    else {
        puts("-----");
        puts("Error in parsing String");
        return 1;
    }
}

// Grammar rule: E -> T E'
int E() {
    printf("%-16s E -> T E'\n", cursor);
    if (T()) { // Call non-terminal T
```



```
        if (Edash()) // Call non-terminal E'
            return SUCCESS;
        else
            return FAILED;
    }
else
    return FAILED;
}

// Grammar rule: E' -> + T E' | $
int Edash() {
    if (*cursor == '+') {
        printf("%-16s E' -> + T E'\n", cursor);
        cursor++;
        if (T()) { // Call non-terminal T
            if (Edash()) // Call non-terminal E'
                return SUCCESS;
            else
                return FAILED;
        }
        else
            return FAILED;
    }
    else {
        printf("%-16s E' -> $\n", cursor);
        return SUCCESS;
    }
}
```



```
    }  
}  
  
// Grammar rule: T -> F T'  
int T() {  
    printf("%-16s T -> F T'\n", cursor);  
    if (F()) { // Call non-terminal F  
        if (Tdash()) // Call non-terminal T'  
            return SUCCESS;  
        else  
            return FAILED;  
    }  
    else  
        return FAILED;  
}  
  
// Grammar rule: T' -> * F T' | $  
int Tdash() {  
    if (*cursor == '*') {  
        printf("%-16s T' -> * F T'\n", cursor);  
        cursor++;  
        if (F()) { // Call non-terminal F  
            if (Tdash()) // Call non-terminal T'  
                return SUCCESS;  
            else  
                return FAILED;  
        }  
    }  
}
```



```
    }
    else
        return FAILED;
}
else {
    printf("%-16s T' -> $\n", cursor);
    return SUCCESS;
}
}

// Grammar rule: F -> ( E ) | i
int F() {
    if (*cursor == '(') {
        printf("%-16s F -> ( E )\n", cursor);
        cursor++;
        if (E()) { // Call non-terminal E
            if (*cursor == ')') {
                cursor++;
                return SUCCESS;
            }
            else
                return FAILED;
        }
        else
            return FAILED;
    }
}
```



```
else if (*cursor == 'i') {  
    printf("%-16s F -> i\n", cursor);  
    cursor++;  
    return SUCCESS;  
}  
else  
    return FAILED;  
}
```

Output:

Enter the string

i + (i * i)

Input	Action
-------	--------

i	E -> T E'
---	-----------

i	T -> F T'
---	-----------

i	F -> i
---	--------

	T' -> \$
--	----------

	E' -> \$
--	----------

String is successfully parsed



Conclusion:

In conclusion, the Recursive Descent Parser program effectively demonstrates the parsing process based on the assumed grammar. By recursively descending through the grammar rules, the program efficiently analyzes input strings to determine whether they conform to the specified grammar.

The output presented by the program provides valuable insights into the acceptance or rejection of input strings based on the grammar rules. This feedback is crucial for understanding the behavior of the parser and its ability to accurately distinguish between valid and invalid strings within the defined language.